

PROJECT PROCESS REPORT

LandingPageSaaS

Full-Stack Landing Page & Compliance Widget Builder

Stack: React 18 · Node.js / Express · MongoDB (Mongoose) · OpenAI (optional)

Prepared for: Cowork handoff / internal documentation

Date: 2026-07-04

Scope: Full architecture, business logic, API reference, and database-migration summary (SQLite ! MongoDB)

1. Project Overview

LandingPageSaaS is an internal SaaS tool that lets a logged-in user generate, customize, and export three kinds of assets from a dashboard:

- **Landing pages** — AI-assisted "Create LP for Desktop" blog-style landing pages with a matching stock hero image.
- **Compliance widgets** — GDPR/CCPA cookie-consent banners and full-page age-verification gates, each with AI-generated copy and styling.
- **Email capture / newsletter widgets** — embeddable signup forms whose captured emails can be viewed, exported (XLSX/CSV), and optionally mirrored to a connected Google Sheet.

Every generated asset is saved server-side as a "page" record, downloadable as a self-contained HTML/ZIP bundle for the user to drop into their own site. An admin panel on top of the same API lets an administrator manage users, feature flags, the global Google Sheet integration, and recover orphaned data.

1.1 Tech Stack

Layer	Technology
Frontend	React 18, React Router 6, Tailwind CSS, Axios, JSZip, file-saver
Backend	Node.js, Express 4, JWT (jsonwebtoken), bcryptjs, Helmet, Morgan, express-validator
Database	MongoDB via Mongoose 8 (models: User, Page, Template, Email, Setting)
AI / Content	OpenAI API (gpt-3.5-turbo / gpt-4o-mini) with deterministic offline fallbacks
Images	Pexels API (optional) or a keyless LoremFlickr fallback
Email	Nodemailer (SMTP) with local HTML templates
Exports	ExcelJS (XLSX), CSV, Archiver (ZIP)

1.2 Recent Change — Database Migration

The backend previously ran on an embedded SQLite database (better-sqlite3) that required zero setup. As part of this report, the data layer was migrated to MongoDB/Mongoose to align with the requested React + Node + MongoDB stack. Key implications:

- **A running MongoDB instance is now required** (local mongod, Docker, or a MongoDB Atlas cluster) — set via MONGODB_URI.
- **Record IDs changed** from auto-increment integers to MongoDB ObjectIds (24-character hex strings). No frontend code depended on numeric IDs, so this required no frontend changes.
- **SQL queries were replaced** with Mongoose model calls (find / findOne / create / findOneAndUpdate / aggregate) across every controller.
- **database/schema.sql** is now a historical reference only; the live schema lives in backend/models/*.js.

2. Folder Structure

```

LandingPageSaaS/
% % % frontend/
% % % % src/
% % % % % components/
% % % % %
% % % % % pages/
% % % % %
% % % % % context/AuthContext.jsx JWT auth state (user, login, logout)
% % % % % utils/ api.js (Axios+JWT), templates.js, zipHelper.js
% % % % % App.jsx Route table
% % % % % package.json

% % % backend/
% % % % config/
% % % % % db.js
% % % % % jwt.js
% % % % % models/
% % % % % controllers/
% % % % %
% % % % % middleware/
% % % % %
% % % % % routes/
% % % % % services/
% % % % % server.js

% % % templates/
% % % database/schema.sql
% % % start.bat
  
```

React + Tailwind client

Navbar, Footer, Sidebar, DashboardLayout, ProtectedRoute, Stepper, CookieBanner, AgeVerificationModal

Home, Dashboard, History, Login, Register, Templates, admin/AdminPanel,

modules/{CookiePage, AgeVerification, EmailNewsletter}

Express + MongoDB API

Mongoose connect() + admin/demo-user seeding

signToken / verifyToken

User, Page, Template, Email, Setting

auth, user, page, template, generate, email-capture, admin

auth.middleware (JWT guard),

admin.middleware (role guard),

validate.middleware

one file per resource, mounted in server.js

email.service, image.service, openai.service

App entry point

Standalone HTML: cookie, age-verification, email

Legacy MySQL reference (unused at runtime)

Windows: launches backend + frontend together

3. Data Models (MongoDB / Mongoose)

All five collections use Mongoose timestamps (created_at / updated_at) except Setting, which is a single-purpose key/value store.

3.1 User

Field	Type	Notes
name	String	Display name
email	String	Unique, lowercased — login identifier
password	String	bcrypt hash (12 salt rounds)
role	String enum	"user" "admin" — default "user"
features_enabled	Mixed (object)	Per-user feature flags, e.g. { ai_generation, custom_templates, email_export, analytics }
sheet_webhook	String	Reserved field; per-page override lives on Page instead

3.2 Page

A saved, generated asset (landing page / cookie banner / age gate / newsletter widget).

Field	Type	Notes
user_id	ObjectId !' User	Owner
type	String enum	landing cookie age-verification newsletter other
domain	String	Brand/site name; used as the de-dupe key with type + user
html_content	String	Full generated HTML, served on download/export
sheet_webhook	String	Optional per-page Google Apps Script webhook URL

3.3 Template

Field	Type	Notes
user_id	ObjectId !' User	Owner
name	String	Template label
type	String	landing cookie age-verification email other
content	String	Saved HTML/content body

3.4 Email

One row per (page, subscriber email) pair — captured from a published widget.

Field	Type	Notes
page_id	ObjectId !' Page	Which widget captured it

email	String	Lowercased; unique together with page_id (prevents duplicate signups)
-------	--------	---

3.5 Setting

A generic key/value document used today only for the global Google Sheet webhook URL (key = "global_sheet_webhook").

4. Authentication & Authorization Logic

4.1 Registration & Login Flow

- `POST /api/auth/register` — validates name/email/password (express-validator), lowercases + uniqueness-checks the email, hashes the password with bcrypt (12 rounds), creates the User with default feature flags, and returns a signed JWT + user summary.
- `POST /api/auth/login` — looks up the user by email, compares the password with `bcrypt.compare`, and returns a JWT containing { id, email, role }.
- `GET /api/auth/me` — decodes the bearer token (via middleware) and returns the fresh user record; used by the frontend `AuthContext` on every page load to restore the session from `localStorage`.

4.2 JWT Middleware (`auth.middleware.js`)

Reads the `Authorization: Bearer <token>` header, verifies it with the shared `JWT_SECRET`, and attaches the decoded payload to `req.user`. Requests without a valid token receive 401. All resource routes (pages, templates, generate, email-capture stats/exports, users, admin) sit behind this guard.

4.3 Role Guard (`admin.middleware.js`)

A second, thin middleware layered on top of `auth.middleware` for the `/api/admin/*` routes — rejects with 403 unless `req.user.role === "admin"`.

4.4 Frontend Session Handling

- **`AuthContext.jsx`** stores the JWT in `localStorage`, exposes { user, loading, login, logout }, and calls `GET /api/auth/me` on mount to rehydrate the session.
- **`utils/api.js`** an Axios instance that automatically attaches the bearer token to every request and force-redirects to `/login` on any 401 response.
- **`ProtectedRoute.jsx`** wraps dashboard/admin routes; shows a spinner while the session check is in flight, redirects unauthenticated users to `/login`, and redirects non-admins away from admin-only routes.

5. Core Business Logic by Feature

5.1 Page Save / Upsert Logic (page.controller.js)

A "page" is uniquely identified by the combination (user_id, type, domain). On create, if a page already exists with that combination, its html_content and sheet_webhook are overwritten in place instead of inserting a duplicate row — this gives each project one persistent save slot rather than growing an unbounded history every time the user re-generates the same asset.

5.2 AI Content Generation (generate.controller.js + openai.service.js)

- POST /api/generate — used for cookie banners, age gates, and newsletters. Builds a prompt (or, if a reference screenshot is supplied, a GPT-4o-mini vision prompt) asking the model to return a strict JSON object of copy + style fields (headline, colors, icon, button text, etc). If OPENAI_API_KEY is missing or the call fails, a deterministic offline fallback returns sensible default copy so the builder keeps working with no key configured.
- POST /api/generate/landing — used for the desktop blog-style landing page. Asks GPT-3.5-turbo for an 800–1000 word structured article (title, intro, quote, 5-7 sections), then calls image.service.fetchStockImage() with the AI's suggested image keyword (falling back to industry, then domain) to attach a matching hero photo.

5.3 Stock Images (image.service.js)

If PEXELS_API_KEY is set, performs a single landscape-orientation search against the Pexels API. Otherwise falls back to LoremFlickr, tagging the request with the same keyword so the placeholder image still visually matches the content niche — no signup required for local development.

5.4 Email Capture & Google Sheet Sync (email-capture.controller.js)

- POST /api/emails/capture — public endpoint embedded in published widgets. Verifies the target page exists, stores the (page_id, email) pair (duplicate submissions are silently ignored via the unique index), and if a Google Apps Script webhook is configured — either per-page (Page.sheet_webhook) or globally (Setting "global_sheet_webhook") — fires an unawaited POST to it so a slow/broken sheet integration never blocks the visitor's submission.
- GET /api/emails/stats — aggregates, per page owned by the caller, the total captured-email count (MongoDB \$lookup + \$size in an aggregation pipeline).
- GET /api/emails/:pageId/export/xlsx and /export/csv — owner-scoped exports of the subscriber list; both accept the JWT either as a normal header or as a ?token= query parameter so the frontend can trigger them via a plain window.open() download link.

5.5 Admin Panel (admin.controller.js)

- User management — list/create/update/delete users, and toggle each user's features_enabled flags (controls which builder modules a given account can access).
- Platform stats — total users / pages / captured emails.
- Global Google Sheet — get/set the shared Apps Script webhook URL used as the fallback destination when a page has no per-page override; validated to be an https://script.google.com/... URL before saving.
- Orphan-page recovery — finds Page documents whose user_id no longer matches any existing User (e.g. after a user was deleted) and lets an admin bulk-reassign them to a chosen (still-existing) user's email — a data-integrity safety net.

5.6 Downloads & Exports

- **Page download (GET /api/pages/:id/download)** streams the saved html_content back as a ZIP (index.html) using Archiver.

- **Frontend zipHelper.js** bundles generated assets client-side with JSZip + file-saver before/instead of hitting the backend, for instant local downloads.
- **Subscriber export** ExcelJS builds a formatted .xlsx workbook; the CSV path streams a plain comma-joined string — both scoped to pages the requesting user owns.

5.7 Transactional Email (email.service.js)

Loads the matching HTML file from templates/email/<name>.html, does a simple {{variable}} string replacement, and sends it via Nodemailer using the configured SMTP_* credentials. Used for welcome and password-reset style notifications.

6. Full API Reference

Base URL: `http://localhost:5000` (configurable via `PORT / REACT_APP_API_URL`). All routes below are prefixed with `/api` unless noted. "JWT" = requires Authorization: Bearer <token>.

6.1 Auth — `/api/auth`

Method	Path	Auth	Description
POST	<code>/register</code>	—	Register a new user, returns JWT + user
POST	<code>/login</code>	—	Login, returns JWT + user
GET	<code>/me</code>	JWT	Get the current user from the token

6.2 Users — `/api/users`

Method	Path	Auth	Description
GET	<code>/me</code>	JWT	Get own profile
PUT	<code>/me</code>	JWT	Update name / password
DELETE	<code>/me</code>	JWT	Delete own account
GET	<code>/me/sheet</code>	JWT	Read-only: the admin-configured global Sheet webhook

6.3 Pages — `/api/pages`

Method	Path	Auth	Description
GET	<code>/</code>	JWT	List own pages
GET	<code>/:id</code>	JWT	Get one page
GET	<code>/:id/download</code>	JWT	Download as ZIP
POST	<code>/</code>	JWT	Create (or upsert by user+type+domain)
PUT	<code>/:id</code>	JWT	Update a page
DELETE	<code>/:id</code>	JWT	Delete a page

6.4 Templates — `/api/templates`

Method	Path	Auth	Description
GET	<code>/</code>	JWT	List templates
GET	<code>/:id</code>	JWT	Get one template
POST	<code>/</code>	JWT	Create a template
PUT	<code>/:id</code>	JWT	Update a template
DELETE	<code>/:id</code>	JWT	Delete a template

6.5 Generate — /api/generate

Method	Path	Auth	Description
POST	/	JWT	AI copy for cookie / age-gate / newsletter
POST	/landing	JWT	AI blog-style landing page + stock image

6.6 Email Capture — /api/emails

Method	Path	Auth	Description
POST	/capture	—	Public: capture a subscriber email
GET	/stats	JWT	Per-page capture counts
GET	/:pageId	JWT	List captured emails
GET	/:pageId/export/xlsx	JWT/?token	Export subscribers as Excel
GET	/:pageId/export/csv	JWT/?token	Export subscribers as CSV

6.7 Admin — /api/admin (role: admin)

Method	Path	Description
GET	/stats	Platform-wide counts
GET	/users	List all users
POST	/users	Create a user
PUT	/users/:id	Update a user
DELETE	/users/:id	Delete a user
PUT	/users/:id/features	Toggle a user's feature flags
GET	/sheet	Get global Sheet webhook
PUT	/sheet	Set global Sheet webhook
GET	/orphan-pages	List pages with no owning user
POST	/recover-pages	Reassign orphaned pages to a user

6.8 Misc

Method	Path	Description
GET	/health	Health check — { status, ts }

7. Frontend Architecture

7.1 Routing (App.jsx)

React Router 6 with three route groups: public (/login, /register), a protected /dashboard shell (DashboardLayout + Sidebar) nesting Dashboard, CookiePage, AgeVerification, EmailNewsletter and History, and a protected + admin-only /admin shell nesting AdminPanel. Unknown paths and the root path redirect to /login (this is an internal tool, not a public marketing site).

7.2 State & Data Flow

- **AuthContext** the single source of truth for the logged-in user; consumed via the useAuth() hook.
- **api.js** central Axios client — every component imports this instead of calling axios directly, so JWT attachment and 401-redirect are handled in exactly one place.
- **Module pages (CookiePage, AgeVerification, EmailNewsletter)** each drives a step-by-step wizard (Stepper.jsx) that collects inputs !' calls /api/generate !' previews the AI result !' saves via /api/pages !' offers a ZIP download.
- **History.jsx** lists previously saved pages (GET /api/pages) with links to re-open, re-download, or view captured emails/stats.
- **AdminPanel.jsx** the admin-only surface over /api/admin/* — user table, feature toggles, stats, and orphan-page recovery.

8. Environment Variables & Setup

8.1 Backend (.env)

Variable	Purpose
MONGODB_URI	Required. MongoDB connection string (local or Atlas)
PORT	Server port, default 5000
JWT_SECRET / JWT_EXPIRES_IN	Token signing secret + TTL (e.g. 7d)
OPENAI_API_KEY	Enables real AI generation; offline fallback used if unset
PEXELS_API_KEY	Enables real stock photos; keyless fallback used if unset
SMTP_HOST / PORT / USER / PASS, EMAIL_FROM	Outgoing transactional email
SEED_ADMIN_EMAIL / SEED_ADMIN_PASSWORD / SEED_USER_PASSWORD	Override the two seeded demo logins
CLIENT_URL	Frontend origin, used for CORS

8.2 Frontend (.env)

REACT_APP_API_URL — backend base URL, default `http://localhost:5000`.

8.3 Local Setup Steps

- 1. Provision MongoDB — install locally, run via Docker, or create a free MongoDB Atlas cluster.
- 2. `cd backend && npm install`, then `cp .env.example .env` and set `MONGODB_URI`.
- 3. `cd frontend && npm install`.
- 4. `cd backend && npm run dev` (`http://localhost:5000`) and, in a second terminal, `cd frontend && npm start` (`http://localhost:3000`). On Windows, `start.bat` launches both.
- 5. On first successful DB connection, an admin account and a demo user account are auto-seeded (see `backend/config/db.js`) — log in with those to explore the dashboard.

9. Verification Status & Known Limitations

9.1 What was verified in this environment

- All backend files (models, config, controllers, routes, server.js) pass node --check with no syntax errors.
- npm install completed cleanly after swapping better-sqlite3 for mongoose in backend/package.json.
- Frontend source was scanned for numeric-ID assumptions (parseInt/Number on ids); none were found, so switching from integer IDs to MongoDB ObjectIds required no frontend changes.

9.2 Not yet verified — requires a live MongoDB instance

- This development environment has no local MongoDB server, Docker, or Atlas credentials available, so the full request/response cycle (register !' login !' generate !' save !' export) has not been exercised end-to-end against the new database layer.
- Recommended next step: point MONGODB_URI at a real MongoDB instance, run npm run dev, and walk through registration, page creation, email capture, and the admin panel once to confirm behavior matches this report before considering the migration production-ready.

10. Summary

LandingPageSaaS is a JWT-authenticated React + Express application for generating and managing compliance/marketing micro-sites, now running on a MongoDB/Mongoose data layer. The system cleanly separates concerns: React handles wizard-style UX and session state, Express controllers own all business rules (upsert-by-project, feature flags, orphan recovery, webhook fan-out), and Mongoose models define the five collections that back everything. AI generation and stock imagery both degrade gracefully to offline defaults when no API key is configured, so the app is fully usable without any external service besides MongoDB itself.